

OBSERVATION TASK 3

Course :	Full Stack Web Development
Course Code:	23CM4121
Observation task no.:	3
Student Name:	R. Srivalli
Roll No.:	A24126552108

1. TITLE

Interactive To-Do List using DOM and Event Listeners

2. AIM

To develop an **Interactive To-Do List** using JavaScript, DOM manipulation, and event listeners that allows users to **add tasks dynamically, mark tasks as completed, and delete tasks** from the webpage while displaying an appropriate message when the task list is empty.

3. OBJECTIVE

- To understand **DOM (Document Object Model) manipulation** in JavaScript.
 - To select and access HTML elements using JavaScript.
 - To dynamically create and add new task elements to a webpage.
 - To implement **event listeners** for handling user actions.
 - To allow users to add tasks through a text input and button.
 - To mark tasks as **completed** using click events and visual styling.
 - To delete individual tasks dynamically from the webpage.
 - To display an appropriate **empty-list message** when no tasks are available.
 - To develop an interactive and user-friendly webpage using **JavaScript and DOM manipulation**.
-

4. THEORY

The **Document Object Model (DOM)** represents an HTML webpage as a hierarchy of elements that can be accessed and modified using JavaScript. Methods such as `getElementById()` and `querySelectorAll()` are used to select and access webpage elements.

JavaScript's createElement() and appendChild() methods allow new task elements to be created and added dynamically to the webpage. **Event listeners** handle user actions such as submitting the task form and clicking the Complete or Delete buttons.

The classList.toggle() method is used to change the task's completed state, while remove() deletes a task from the DOM. The application also checks the number of tasks and displays an appropriate message when the list is empty.

Thus, the program demonstrates **DOM selection, dynamic element creation, DOM modification, event handling, and interactive webpage development**

5. ALGORITHM / PROCEDURE

1. Create an HTML page containing a **task input box**, **Add Task** button, and task list area.
 2. Select the required HTML elements using DOM methods such as getElementById().
 3. Add a **submit event listener** to the task form.
 4. Read and trim the task entered by the user.
 5. If the input is empty, display a validation message and stop the operation.
 6. Create a new task element dynamically using createElement().
 7. Create the task number, task text, **Complete** button, and **Delete** button.
 8. Add click event listeners to the Complete and Delete buttons.
 9. Use classList.toggle() to mark or unmark a task as completed.
 10. Use remove() to delete the selected task from the webpage.
 11. Update the task counter and empty-list message after every addition or deletion.
 12. Clear the input field and display the updated task list.
-

6. PROGRAM

A : Index.html

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Task Board | Interactive To-Do List</title>

  <!-- External CSS file -->
  <link rel="stylesheet" href="styleobstask3.css">
</head>

<body>
```

```

<main class="page-container">

  <!-- Header section -->
  <header class="hero-section">

    <div class="brand-row">
      <div class="brand-mark">✓</div>

      <div>
        <p class="brand-name">TASK BOARD</p>
        <p class="brand-tag">JAVASCRIPT • DOM</p>
      </div>
    </div>

    <div class="hero-content">
      <p class="eyebrow">OBSERVATION TASK 3</p>

      <h1>
        Stay organized.<br>
        <span>Get things done.</span>
      </h1>

      <p class="hero-description">
        Add your tasks, mark them complete, and remove
        them whenever they are no longer needed.
      </p>
    </div>

  </header>

  <!-- Main application area -->
  <section class="todo-section">

    <!-- Add task area -->
    <div class="task-input-box">

      <div class="section-heading">
        <div>
          <p class="small-label">YOUR WORKSPACE</p>
          <h2>Create a task</h2>
        </div>

        <span class="task-counter" id="taskCounter">0 TASKS</span>
      </div>

      <!-- Form is used so Enter can also add a task -->
      <form id="taskForm" class="task-form">

        <input
          type="text"
          id="taskInput"

```

```

        placeholder="Enter a task..."
        autocomplete="off"
    >

    <button type="submit" id="addTaskButton">
        <span>+</span>
        Add Task
    </button>

</form>

<p id="inputMessage" class="input-message"></p>

</div>

<!-- Task list area -->
<div class="task-list-box">

    <div class="list-heading">
        <div>
            <p class="small-label">TODAY</p>
            <h2>Your tasks</h2>
        </div>
    </div>

    <!-- JavaScript dynamically adds tasks here -->
    <div id="taskList" class="task-list">

        <!-- Empty message shown when there are no tasks -->
        <div id="emptyMessage" class="empty-message">

            <div class="empty-icon">✓</div>

            <h3>All clear</h3>

            <p>
                No tasks available.<br>
                Add a new task to get started.
            </p>

        </div>

    </div>

</div>

</section>

<!-- Footer -->
<footer class="footer">

```

```

    <p>
      Interactive To-Do List
    <span>•</span>
    DOM & Event Listeners
  </p>

</footer>

</main>

<!-- External JavaScript file -->
<script src="scriptobstask3.js"></script>

</body>

</html>

```

B : style.css

```

/* =====
OBSERVATION TASK 3
Interactive To-Do List
CSS Styling
===== */

/* ----- Basic Reset ----- */

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

/* ----- Page ----- */

body {
  min-height: 100vh;
  font-family: Arial, Helvetica, sans-serif;
  background: #eeeeae;
  color: #171717;
  padding: 35px 20px;
}

/* ----- Main Container ----- */

.page-container {
  width: 100%;
  max-width: 1050px;
  margin: 0 auto;
  background: #f8f6f1;

```

```
border: 1px solid #d8d3c9;
box-shadow: 0 20px 60px rgba(0, 0, 0, 0.08);
}

/* =====
HERO SECTION
===== */

.hero-section {
  background: #171717;
  color: #ffffff;
  padding: 42px 50px 48px;
}

/* ----- Brand ----- */

.brand-row {
  display: flex;
  align-items: center;
  gap: 12px;
  margin-bottom: 65px;
}

.brand-mark {
  width: 38px;
  height: 38px;
  border-radius: 50%;
  display: flex;
  align-items: center;
  justify-content: center;

  background: #d7b56d;
  color: #171717;

  font-size: 20px;
  font-weight: bold;
}

.brand-name {
  font-size: 13px;
  font-weight: bold;
  letter-spacing: 2px;
}

.brand-tag {
  margin-top: 3px;
  color: #aaa69d;
  font-size: 9px;
  letter-spacing: 1.5px;
}
```

```

/* ----- Hero Content ----- */

.eyebrow {
  color: #d7b56d;
  font-size: 11px;
  font-weight: bold;
  letter-spacing: 2px;
  margin-bottom: 14px;
}

.hero-content h1 {
  max-width: 700px;
  font-size: clamp(38px, 6vw, 65px);
  line-height: 1;
  letter-spacing: -2.5px;
  font-weight: 700;
}

.hero-content h1 span {
  color: #d7b56d;
}

.hero-description {
  max-width: 550px;
  margin-top: 22px;
  color: #b8b5ae;
  font-size: 15px;
  line-height: 1.7;
}

/* =====
TODO SECTION
===== */

.todo-section {
  padding: 45px 50px 50px;
}

/* ----- Section Heading ----- */

.section-heading,
.list-heading {
  display: flex;
  align-items: flex-end;
  justify-content: space-between;
  gap: 20px;
}

.small-label {
  color: #8b8579;
  font-size: 10px;
}

```

```
font-weight: bold;
letter-spacing: 1.8px;
margin-bottom: 7px;
}

.section-heading h2,
.list-heading h2 {
  font-size: 25px;
  letter-spacing: -0.5px;
}

.task-counter {
  border: 1px solid #d4cec2;
  padding: 8px 12px;
  border-radius: 30px;

  color: #756f64;
  font-size: 10px;
  font-weight: bold;
  letter-spacing: 1px;
}

/* =====
TASK INPUT
===== */

.task-input-box {
  padding-bottom: 42px;
  border-bottom: 1px solid #ddd8cf;
}

.task-form {
  display: flex;
  gap: 12px;
  margin-top: 24px;
}

.task-form input {
  flex: 1;
  min-width: 0;

  padding: 17px 18px;

  border: 1px solid #d2cdc3;
  border-radius: 4px;

  background: #ffffff;
  color: #222222;

  font-size: 14px;
  outline: none;
```

```
    transition: 0.2s ease;
}

.task-form input::placeholder {
    color: #aaa59c;
}

.task-form input:focus {
    border-color: #171717;
    box-shadow: 0 0 3px rgba(215, 181, 109, 0.18);
}

/* ----- Add Button ----- */

#addTaskButton {
    min-width: 135px;

    border: none;
    border-radius: 4px;

    background: #171717;
    color: #ffffff;

    font-size: 13px;
    font-weight: bold;

    cursor: pointer;

    transition: 0.2s ease;
}

#addTaskButton span {
    color: #d7b56d;
    font-size: 18px;
    margin-right: 4px;
}

#addTaskButton:hover {
    background: #2d2d2d;
    transform: translateY(-1px);
}

#addTaskButton:active {
    transform: translateY(0);
}

/* ----- Input Message ----- */

.input-message {
    min-height: 18px;
```

```

margin-top: 8px;

color: #a55d45;
font-size: 12px;
}

/* =====
TASK LIST
===== */

.task-list-box {
padding-top: 40px;
}

.task-list {
margin-top: 22px;
}

/* =====
EMPTY MESSAGE
===== */

.empty-message {
padding: 55px 20px;
border: 1px dashed #cec8bd;
text-align: center;
}

.empty-icon {
width: 48px;
height: 48px;

margin: 0 auto 15px;

display: flex;
align-items: center;
justify-content: center;

border-radius: 50%;

background: #e9dfca;
color: #856a32;

font-size: 21px;
font-weight: bold;
}

.empty-message h3 {
font-size: 18px;
margin-bottom: 7px;
}

```

```

.empty-message p {
  color: #888278;
  font-size: 13px;
  line-height: 1.7;
}

/* =====
DYNAMIC TASK ITEM
===== */

.task-item {
  display: flex;
  align-items: center;
  gap: 15px;

  padding: 17px 18px;

  margin-bottom: 10px;

  background: #ffffff;
  border: 1px solid #ddd8cf;

  transition:
    transform 0.2s ease,
    box-shadow 0.2s ease,
    background 0.2s ease;
}

.task-item:hover {
  transform: translateX(3px);
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.06);
}

/* ----- Task Number / Indicator ----- */

.task-indicator {
  width: 30px;
  height: 30px;

  flex-shrink: 0;

  display: flex;
  align-items: center;
  justify-content: center;

  border: 1px solid #c9c2b6;
  border-radius: 50%;

  color: #777168;
  font-size: 11px;

```

```
    font-weight: bold;
}

/* ----- Task Text ----- */

.task-text {
    flex: 1;
    min-width: 0;

    color: #292724;
    font-size: 14px;
    line-height: 1.5;

    overflow-wrap: anywhere;
}

/* ----- Buttons ----- */

.task-actions {
    display: flex;
    gap: 7px;
    flex-shrink: 0;
}

.complete-button,
.delete-button {
    border: none;
    padding: 9px 12px;

    border-radius: 3px;

    font-size: 11px;
    font-weight: bold;

    cursor: pointer;

    transition: 0.2s ease;
}

.complete-button {
    background: #eee8dc;
    color: #5f5138;
}

.complete-button:hover {
    background: #d7b56d;
    color: #171717;
}

.delete-button {
    background: #f1eeee;
```

```

        color: #8b5147;
    }

.delete-button:hover {
    background: #dfc8c3;
    color: #63372f;
}

/* =====
COMPLETED TASK
===== */

.task-item.completed {
    background: #f1efe9;
    opacity: 0.72;
}

.task-item.completed .task-indicator {
    background: #171717;
    border-color: #171717;
    color: #d7b56d;
}

.task-item.completed .task-text {
    color: #77736c;
    text-decoration: line-through;
    text-decoration-thickness: 1.5px;
}

.task-item.completed .complete-button {
    background: #171717;
    color: #d7b56d;
}

/* =====
FOOTER
===== */

.footer {
    padding: 20px 50px;

    border-top: 1px solid #ddd8cf;

    background: #f1eee8;

    text-align: center;

    color: #918b80;
    font-size: 10px;
    letter-spacing: 0.8px;
}

```

```
.footer span {
  margin: 0 6px;
  color: #b4965d;
}

/* =====
RESPONSIVE DESIGN
===== */

@media (max-width: 700px) {

  body {
    padding: 15px;
  }

  .hero-section {
    padding: 32px 25px 38px;
  }

  .brand-row {
    margin-bottom: 45px;
  }

  .hero-content h1 {
    font-size: 43px;
    letter-spacing: -1.8px;
  }

  .todo-section {
    padding: 32px 25px 35px;
  }

  .task-form {
    flex-direction: column;
  }

  #addTaskButton {
    min-height: 48px;
  }

  .task-item {
    align-items: flex-start;
    flex-wrap: wrap;
  }

  .task-text {
    padding-top: 5px;
  }

  .task-actions {
```

```
width: 100%;
margin-left: 45px;
}

.complete-button,
.delete-button {
  flex: 1;
}

.footer {
  padding: 18px 25px;
}
}

@media (max-width: 450px) {

  body {
    padding: 0;
  }

  .page-container {
    border: none;
  }

  .hero-section {
    padding: 28px 20px 35px;
  }

  .todo-section {
    padding: 28px 20px 32px;
  }

  .section-heading h2,
  .list-heading h2 {
    font-size: 21px;
  }

  .task-counter {
    font-size: 8px;
    padding: 7px 9px;
  }

  .hero-content h1 {
    font-size: 36px;
  }

  .hero-description {
    font-size: 13px;
  }

  .task-item {
```

```

padding: 14px;
}

.task-actions {
margin-left: 0;
}

.footer {
padding: 17px 20px;
}
}

```

C : script.js

```

/* =====
OBSERVATION TASK 3
Interactive To-Do List
JavaScript + DOM + Event Listeners
===== */

/* ----- Select DOM elements ----- */

const taskForm = document.getElementById("taskForm");
const taskInput = document.getElementById("taskInput");
const taskList = document.getElementById("taskList");
const emptyMessage = document.getElementById("emptyMessage");
const taskCounter = document.getElementById("taskCounter");
const inputMessage = document.getElementById("inputMessage");

/* ----- Task numbering ----- */

let taskNumber = 0;

/* =====
UPDATE TASK COUNTER
===== */

function updateTaskCounter() {

const tasks = taskList.querySelectorAll(".task-item");
const totalTasks = tasks.length;

if (totalTasks === 1) {
taskCounter.textContent = "1 TASK";
} else {
taskCounter.textContent = `${totalTasks} TASKS`;
}
}

/* =====

```

```

UPDATE EMPTY MESSAGE
===== */

function updateEmptyMessage() {

    const tasks = taskList.querySelectorAll(".task-item");

    if (tasks.length === 0) {
        emptyMessage.style.display = "block";
    } else {
        emptyMessage.style.display = "none";
    }
}

/* =====
CREATE TASK
===== */

function createTask(taskText) {

    taskNumber++;

    /* ----- Main task element ----- */

    const taskItem = document.createElement("div");
    taskItem.classList.add("task-item");

    /* ----- Task number ----- */

    const indicator = document.createElement("div");
    indicator.classList.add("task-indicator");
    indicator.textContent = taskNumber;

    /* ----- Task text ----- */

    const textElement = document.createElement("p");
    textElement.classList.add("task-text");
    textElement.textContent = taskText;

    /* ----- Button container ----- */

    const actions = document.createElement("div");
    actions.classList.add("task-actions");

    /* ----- Complete button ----- */

    const completeButton = document.createElement("button");

    completeButton.type = "button";
    completeButton.classList.add("complete-button");
    completeButton.textContent = "Complete";

```

```

/* ----- Delete button ----- */

const deleteButton = document.createElement("button");

deleteButton.type = "button";
deleteButton.classList.add("delete-button");
deleteButton.textContent = "Delete";

/* =====
COMPLETE BUTTON
===== */

completeButton.addEventListener("click", function () {

    taskItem.classList.toggle("completed");

    if (taskItem.classList.contains("completed")) {

        completeButton.textContent = "Completed";
        indicator.textContent = "✓";

    } else {

        completeButton.textContent = "Complete";
        indicator.textContent = taskNumber;

    }

});

/* =====
DELETE BUTTON
===== */

deleteButton.addEventListener("click", function () {

    taskItem.remove();

    updateTaskCounter();
    updateEmptyMessage();

});

/* ----- Build task ----- */

actions.appendChild(completeButton);
actions.appendChild(deleteButton);

taskItem.appendChild(indicator);
taskItem.appendChild(textElement);

```

```

taskItem.appendChild(actions);

/* ----- Add task to webpage ----- */

taskList.appendChild(taskItem);

/* ----- Update interface ----- */

updateTaskCounter();
updateEmptyMessage();
}

/* =====
FORM SUBMIT EVENT
===== */

taskForm.addEventListener("submit", function (event) {

    // Prevent page refresh
    event.preventDefault();

    // Get entered task
    const taskText = taskInput.value.trim();

    /* ----- Validate input ----- */

    if (taskText === "") {

        inputMessage.textContent =
            "Please enter a task before adding.";

        taskInput.focus();

        return;
    }

    /* ----- Clear error message ----- */

    inputMessage.textContent = "";

    /* ----- Create task ----- */

    createTask(taskText);

    /* ----- Clear input ----- */

    taskInput.value = "";

    taskInput.focus();

});

```

7. CODE EXPLANATION

S.No.	Code / Component	Explanation
1	<code>getElementById()</code>	Selects the form, input field, task list, counter, and empty-message elements from the DOM.
2	<code>let taskNumber = 0</code>	Maintains the numbering of tasks added to the list.
3	<code>updateTaskCounter()</code>	Counts the existing task elements and updates the task counter.
4	<code>querySelectorAll(".task-item")</code>	Selects all dynamically created task items from the task list.
5	<code>updateEmptyMessage()</code>	Displays the empty message when there are no tasks and hides it when tasks exist.
6	<code>createTask(taskText)</code>	Creates a complete task item dynamically using the entered task text.
7	<code>document.createElement()</code>	Dynamically creates elements such as the task container, text, and buttons.
8	<code>classList.add()</code>	Assigns CSS classes to dynamically created elements for styling.
9	<code>textContent</code>	Inserts the task number, task text, and button labels into the created elements.
10	<code>appendChild()</code>	Builds the task structure and adds it to the webpage.
11	<code>addEventListener("click", ...)</code>	Handles clicks on the Complete and Delete buttons.
12	<code>classList.toggle("completed")</code>	Adds or removes the completed class to change the task's completion state.
13	<code>classList.contains("completed")</code>	Checks whether the selected task is currently completed.
14	<code>taskItem.remove()</code>	Removes the selected task element from the DOM when Delete is clicked.
15	<code>addEventListener("submit", ...)</code>	Handles submission of the task form when Add Task is clicked or Enter is pressed.

16	<code>event.preventDefault()</code>	Prevents the page from refreshing when the form is submitted.
17	<code>taskInput.value.trim()</code>	Reads the entered task and removes unnecessary spaces.
18	<code>if (taskText === "")</code>	Validates the input and prevents an empty task from being added.
19	<code>taskInput.focus()</code>	Places the cursor back in the input field for convenient task entry.
20	<code>updateTaskCounter()</code> / <code>updateEmptyMessage()</code>	Keeps the task count and empty-state message synchronized with the current DOM.

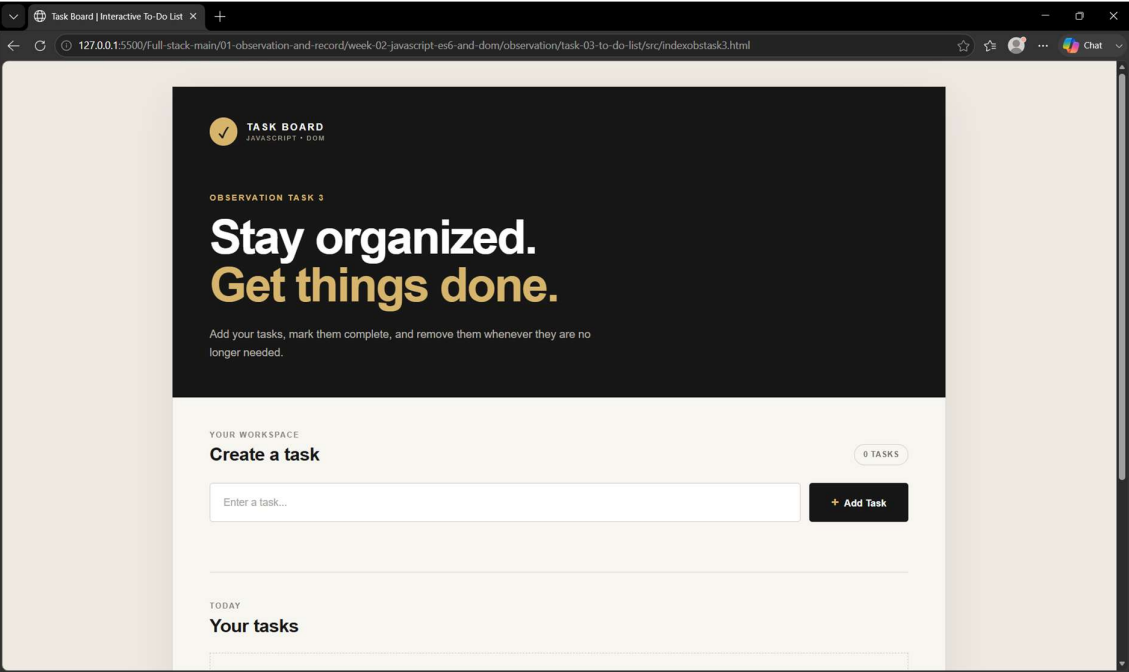
8. TEST CASES AND EXECUTION DATA

Test Case	Input	Action	Expected Result	Status
TC01	No tasks	Open the webpage	"No tasks available" message is displayed	Pass
TC02	Complete Assignment	Click Add Task	Task is dynamically added to the list	Pass
TC03	Prepare for Quiz	Add the task	Second task is displayed with Complete and Delete buttons	Pass
TC04	Practice JavaScript	Add the task	Third task is displayed and task counter updates	Pass
TC05	Existing task	Click Complete	Task is marked completed with a line-through effect	Pass
TC06	Completed task	Click Completed again	Task returns to its active state	Pass
TC07	Existing task	Click Delete	Selected task is removed from the list	Pass
TC08	Empty input	Click Add Task	Validation message is displayed and no task is added	Pass
TC09	Last remaining task	Click Delete	Task is removed and "No tasks available" message reappears	Pass

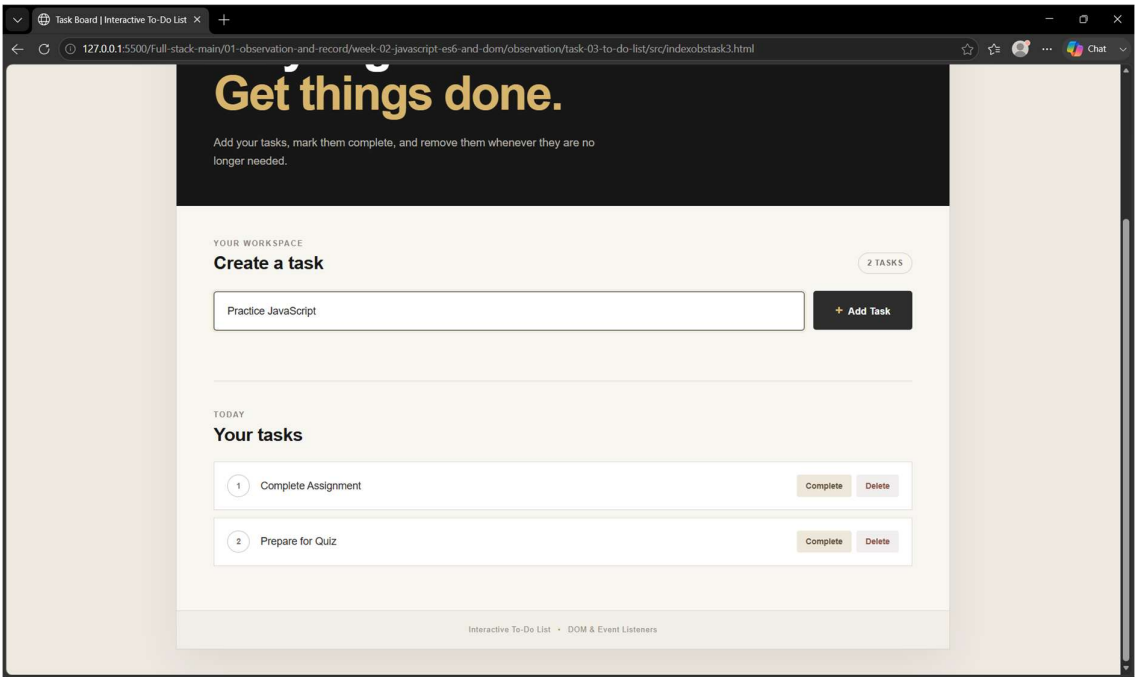
TC10	Valid task entered	Press Enter	Task is added without clicking the button	Pass
------	--------------------	-------------	---	------

9. OUTPUT

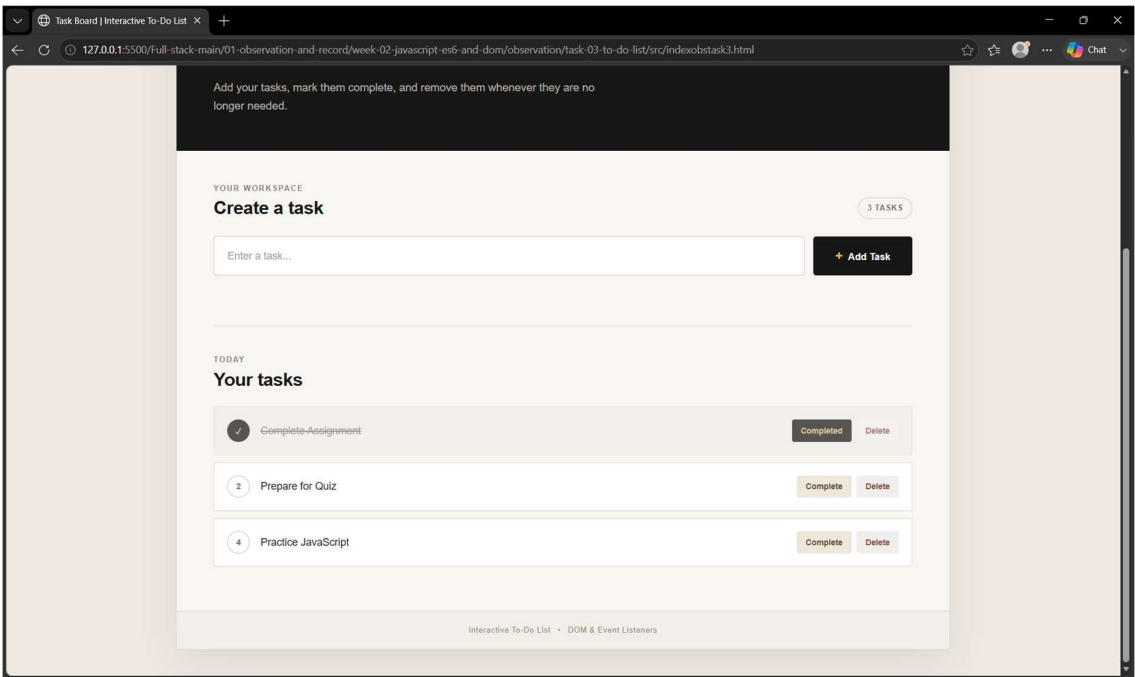
Output 1: Initial To-Do List



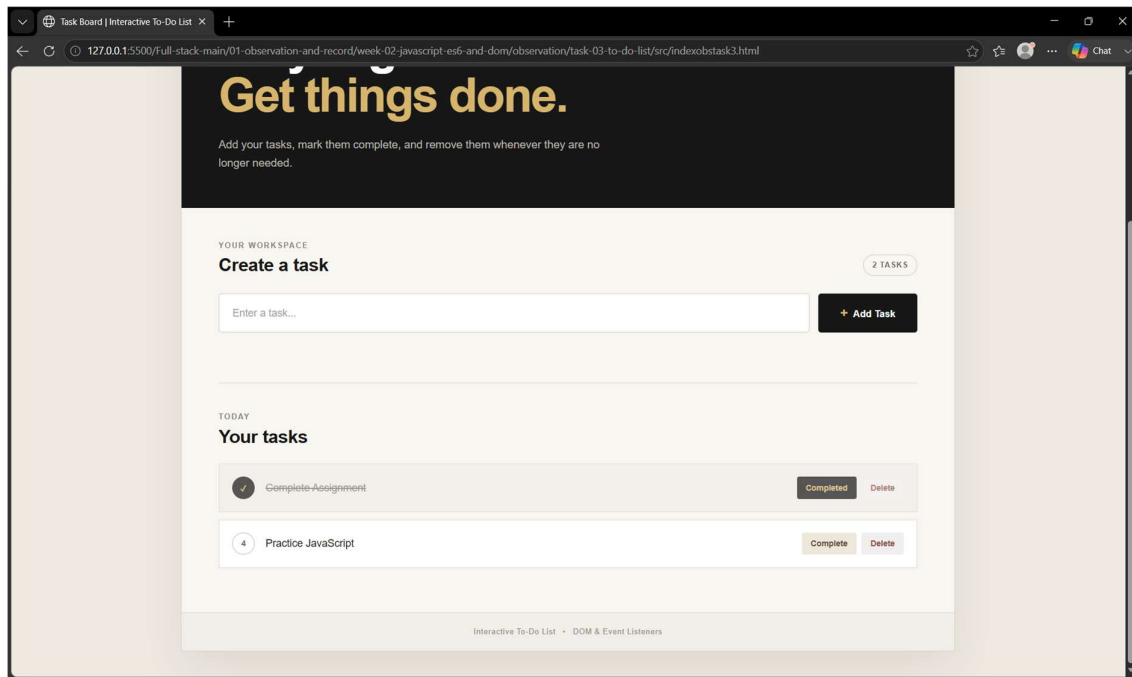
Output 2: Tasks Added



Output 3: Completed Task



Output 4: Deleted Task / Empty List



10. RESULT

The Interactive To-Do List was successfully developed using JavaScript, DOM manipulation, and event listeners. Users can dynamically add tasks, mark tasks as completed, delete tasks, and view an appropriate message when the task list is empty.